Clients Rot, Connections Don't: Degradation, Silent Failures and Refresh in Long-Lived WMI Connections

Author Name
Affiliation

Draft v4.1, 2026-10-24. **[Red: author input]**

Abstract

Windows management agents commonly recycle their WMI connections every few hours on the belief that long-lived connections degrade. We test that belief with a client whose COM usage is verified on every query, which checks results against non-WMI ground truth and attributes resource use to the provider host serving each query. On two hosts, one Windows 10-based and one Windows 11-based, one set of connections served 381,600 and 194,399 queries over 6 h at 100× and 50× a production polling rate (about 25 and 12.5 production-days of queries). Latency, median and 99th percentile, did not change with connection age on either host. Client memory did not grow measurably on the first (0.039 MB/h, 95% CI [−0.007, 0.086]; an upper bound near 7.5 MB/year at the production rate); on the second it grew slowly over 6 h (0.218 MB/h [0.153, 0.282], in steps), well below our pre-set bound and fully reclaimed when the connections closed, and in a 72-hour run it levelled off at about 1.5 MB after 10 h (interim, 32 h). The degradation we could reproduce was client-induced: an unreleased `VARIANT` array leaks about 140 B per read, and cross-apartment calls to the scripting objects cost about 3× the wall-clock time per collection cycle (5–8× the CPU). *Silent* failures matter more: a popular Go COM library ends enumeration without reporting the error, three of the earlier client's 23 queries fail on every poll on both architectures while logged as successes, and `Win32_SystemDriver` omits a registered driver whose binary is missing, depending on the properties selected. Fault injection shows where refresh matters: on both hosts a restart of the WMI service broke every existing connection permanently; reconnecting on the error restored service within seconds, a 1-hour timer after 30 min, and a replica of our earlier client's pool not within the hour, because its health check does not recognise the error. A crash of a provider host cost at most the call it was serving; the connection survived. Agents should invest in client correctness and result verification, and reconnect on errors rather than on a clock.

1 Introduction

Windows Management Instrumentation (WMI) is the standard inventory and telemetry interface on Windows; endpoint agents, monitoring systems and configuration tools poll it continuously. A widespread practice is to treat WMI connections as perishable: agents tear down and recreate their `IWbemServices` connections on a timer or a query budget. The belief is documented in the tooling as well as in operations: a widely used Go WMI package warns of a "memory bug" and advises callers to run queries in a dedicated process and exit after a bounded number of queries [12], and reported client leaks have been traced to unreleased COM objects rather than to connection age [13]. This belief is where we began. While building esinta, our OT endpoint-telemetry agent, we saw its WMI client's memory grow, its latency creep and its calls time out, and took these for connection ageing. We built the probe described here to test that hypothesis against our earlier client, the WMI layer of earlier esinta builds, which issues the 23 queries below and recycles every 6 h or 50,000 queries, the standard refresh-on-timer rule. We expected to confirm that long-lived connections rot; instead the verified client showed the degradation was ours. *Disclosure:* the authors develop esinta commercially; the earlier client studied here and the bugs reported in it are our own, found and fixed between September and October 2026.

Whether those symptoms come from *connection age* or from the *client* has, to our knowledge, not been measured. It is hard to measure: an incorrect client produces exactly these symptoms, the usual metric (memory summed over all `WmiPrvSE.exe` processes) mixes every WMI client on the host, and many WMI failures are silent.

We contribute (1) a method that removes these confounders: a self-tested correct client, per-provider-host attribution, an ETW meter of foreign WMI clients, and non-WMI ground truth (§3); (2) evidence on two architectures that a correct client's connections do not degrade in latency, and that client memory stays small and below a pre-set bound, at accelerated query counts (§4); (3) measurements of two client bugs that produce the folklore symptoms, and a catalogue of silent failures (§5, §6); and (4) a fault-injection comparison of refresh strategies with concrete guidance (§7, §8).

2 Background

A client calls `IWbemServices` in the WMI service (`Winmgmt`), which dispatches each class to a provider. In-process providers run in shared `WmiPrvSE.exe` host processes, one

per hosting model such as NetworkService or LocalService [2]. WMI unloads a provider once it has been idle for its cache-control interval (`ClearAfter`; unloading takes "anywhere from the interval … to twice that interval") [3, 4], and enforces per-host quotas on memory, threads and handles [5]. Queries issued with the return-immediately and forward-only flags (0x30) are semisynchronous: `ExecQuery` returns an enumerator at once, and errors surface later from `IEnumWbemClassObject::Next`, which returns `WBEM_S_FALSE` at the normal end of a result [6]. The scripting objects (`SWbemLocator` and relatives) are apartment-threaded: a call from another apartment is marshalled to the owning single-threaded apartment (STA) [7], and property values arrive as `VARIANT`s that the caller must free with `VariantClear`, which frees strings and arrays [8]. A client that walks a result set through the scripting objects enumerates an `SWbemObjectSet` via `IEnumVARIANT::Next` [19] rather than `IEnumWbemClassObject::Next`; the two share the convention that `S_FALSE` with no object marks the normal end of a result. Throughout, we distinguish the *outer* `HRESULT` a client sees at the call boundary from the *effective* error code: a failure dispatched through `IDispatch::Invoke` is reported to the caller as `DISP_E_EXCEPTION` with the originating WBEM or RPC code carried in the `EXCEPINFO`, so a health check that inspects only the outer code can miss a connection-level fault [18].

3 Method

A correct client. The probe (Go, `go-ole`) gives every connection its own OS thread initialised as an STA, and makes every COM call for that connection, including release, on that thread. Every `VARIANT` is cleared exactly once; a ledger counts acquisitions and releases per query and flags outstanding or double releases. Enumeration uses a strict loop: `S_OK` yields an item, `S_FALSE` with no item ends the result, and any other code is an error recorded with its `HRESULT`, WBEM code and the operation that failed. A query that exceeds its deadline is abandoned on the caller's side and its row is written when WMI eventually answers.

Workload. The catalogue reproduces the 23 queries of our earlier client verbatim (processes, services, drivers, event logs, firewall, Defender, WMI subscriptions, scheduled tasks, Device Guard, TPM, BitLocker), each at its production rate. A query runs only where its namespace exists: on the Windows 10-based host 22 queries in 7 namespaces (BitLocker's namespace is absent; 636 queries/h), on the Windows 11-based host all 23 in 8 namespaces (648 queries/h). The probe holds three connections to `root\cimv2` and one to each other namespace (9 and 10 connections). An acceleration factor k multiplies every rate. A pre-flight calibration on each host (10-minute steps, $k \in \{1, 5, 10, 20, 50, 100\}$) measured probe-induced CPU (probe + all provider hosts + `Winmgmt`, as a percentage of the 2-vCPU host) of 0.4, 1.6, 3.3, 6.3, 15.6 and 29.9% on the Windows 10-based host and 0.6, 3.0, 6.0, 11.6, 29.5 and 58.1% on the Windows 11-based host, with no dropped queries; k_H, the largest k within a 40% budget, is 100 (the top of the ramp) and 50. Both hosts present the same virtual CPU model; RAM and edition differ, so absolute latency and CPU are compared within a host, not across hosts.

Attribution and isolation. Every 10 s the probe samples itself, the `Winmgmt` service host (PID from the service control manager) and each `WmiPrvSE` by PID and creation time, labelled with its account and the provider DLLs it has loaded; this maps each queried class to the host serving it. Nothing here uses WMI. The class-to-host map was the same on both hosts: the CIMWin32, event-log, firewall, scheduled-task, Device Guard, TPM and (where present) BitLocker providers share the NetworkService host, and Defender's provider runs in the LocalService host. A real-time ETW session on `Microsoft-Windows-WMI-Activity` [9] counts operation starts by client PID, so foreign WMI clients are observed rather than assumed absent: idle, it counted 3 foreign operations per hour on the Windows 10-based host (60 minutes) and 16 on the Windows 11-based host (30 minutes, immediately before its runs; an earlier 60-minute baseline that began 15 minutes after a reboot failed at 611/h, all but 28 of them in its first 16 minutes, from post-boot maintenance tasks), against a ceiling of 60/h fixed in the pre-registered design (one foreign operation per minute on average, under 0.2% of the probe's own queries at k_H): a meter window above it is flagged, and an arm with more than 10% of its windows flagged is invalid. During the arms the session was duty-cycled, one minute in fifteen, to limit its own overhead, so foreign activity is sampled; flagged windows are reported in Section 8. Because service-host grouping differs across editions and memory sizes [20], we read from the service control manager whether `Winmgmt` runs in its own `svchost.exe` or shares one: on both hosts it ran alone in its own. We found that `tasklist.exe`, the CIM cmdlets and PowerShell remoting all issue WMI calls; host inspection during runs was restricted to tools measured at zero WMI operations.

Ground truth. Results are compared with non-WMI sources taken immediately before and after each query: the process list (`NtQuerySystemInformation`), the service control manager, and invariants captured at the start. Five silent classes are counted: S1 impossible-empty, S2 truncated (a key present in both truth snapshots is missing), S3 enumeration ended by an error (what a lenient loop would report as success), S4 partial objects, S5 content mismatch.

Testbed and statistics. Two virtual machines with 2 vCPU each, one Windows 10-based (4 GB) and one Windows 11-based (8 GB), both with Defender real-time protection on and no other WMI-heavy software. Both hosts ran the same probe build and analysis code. Growth is the OLS slope of 10 s samples after a 30-minute warm-up with Newey–West standard errors (15-minute lag window) [11]. Pass criteria were fixed before the runs, in a commit of the design

Table 1: ST-1: one set of connections, strategy *none*, 6 h, on each host. All slopes are over the same 5.5-hour window (after the 30-minute warm-up) and are net over it; how growth is distributed within the window is given in the text. CIs are 95%, HAC. Memory in MB $= 10^6$ B.

	Win 10-based	Win 11-based
k (production-days of queries)	100 (25)	50 (12.5)
Queries	381,600	194,399
ok	360,000	183,590
deterministic / transient error[b]	21,600 / 0	10,799 / 6
abandoned	0	0
Silent failures above truth baseline[b]	0	3
Client private bytes (MB/h)	0.039	0.218
	[−0.007, 0.086]	[0.153, 0.282]
per array read (B)	0.06	0.37
	[−0.01, 0.14]	[0.26, 0.48]
array reads	3.48 M	3.24 M
Client handles: range	17	7
Retained after 1 h release (MB)	−0.41	−0.46
NetworkService host (MB/h)	0.085	0.681
	[0.066, 0.103]	[0.495, 0.866]
LocalService host (MB/h)	0.036	0.103[c]
	[0.001, 0.071]	[0.055, 0.152]
Winmgmt host (MB/h)	−0.020	0.182
	[−0.043, 0.003]	[0.129, 0.235]
Latency p99, last vs. first hour[a]	−1.1 to +0.4%	−1.7 to +2.5%
Recycles; induced CPU	0; 30.2%	0; 29.2%

[a] range over the query keys with median latency above 100 ms (six and seven keys). [b] Windows 11-based: six Defender-provider errors in 10 s during a Defender platform update, and three scheduled-task results one task short in 12 s (S2); see Section 6. [c] the host was replaced after 4.4 h; slope of the first host.

(`9dfe8f1`, dated 2026-10-16; the gate script implementing them, `c5e4fcb`, has not changed since, and its data loader last changed in `29b53a0` on 2026-10-17, before any gate result was computed; commit hashes refer to the public repository [1]): client private-byte slope < 1 MB/h and < 3 B per array read (upper 95% bound), handle range ≤ 50, and ≤ 5 MB retained after a 1-hour release test. Memory figures use MB $= 10^6$ bytes throughout.

4 A correct client's connections do not degrade

The self-test arm (strategy *none*: one set of connections held open for the whole run, with the probe's per-query COM ledger active) doubles as our longest correct-client run on each host (Table 1). At $k = 100$ its 6 hours carry the query count of about 25 days at the production rate (12.5 days at $k = 50$). This accelerates the *count* of queries, not the age of the connection and not the provider lifecycle: at these rates the providers never go idle long enough to unload, so the arm contains none of the unload/reload cycles a production client sees every few minutes (below). All client invariants held on every query on both hosts (correct apartment, no outstanding, double or leaked `VARIANT`s, no panics), and both passed the pre-set criteria.

Windows 10-based host. Client memory did not grow measurably. If growth tracks queries, we can rule out at the production rate a client-side leak larger than about 0.86 kB/h (roughly 7.5 MB/year) at 95% confidence. Median latency per query key stayed flat across connection age, from about 2 ms (repository classes) to 230 ms (`MSFT_NetFirewallRule`, 260 rows; first to last hour of the window −2%). The tails were as flat: for the six keys with medians above 100 ms, the 95th and 99th percentiles moved by at most 1.6% from the first to the last hour, no key's 99th percentile rose by more than 1.9 ms, and no successful query took longer than 363 ms against a 30 s deadline (none was abandoned). The `Winmgmt` host was flat (−0.020 MB/h [−0.043, 0.003]). The provider hosts grew mostly early: by thirds of the window, the LocalService host 0.19, 0.01 and −0.03 MB/h, and the NetworkService host, serving six providers, 0.13, 0.07 and 0.02 MB/h [−0.09, 0.13] (0.085 MB/h [0.066, 0.103] over the window, about 0.5 MB in 5.5 h). Six hours cannot distinguish a plateau from slow growth there; the 7-day run will. The single client handle step (339 to 347) coincides with one additional Go runtime thread and is flat before and after. A 7-day run at $k = 100$ on the Windows 10-based host (about 700 production-days of queries per connection, though not of provider reloads; see below) is in progress and will replace this section's age bound.

Windows 11-based host. Latency was again flat with connection age: first-to-last-hour medians moved by less than 0.2% for `MSFT_NetFirewallRule` (743 ms), `Win32_SystemDriver` (289 ms) and `Win32_Service` (225 ms). The full process list is bimodal on this host (in every half hour, half its calls took about 136 ms and half about 370 ms), so its median jumps between the modes; its quartiles did not move. For the seven keys with medians above 100 ms the 95th and 99th percentiles moved by at most 2.5% from the first to the last hour, and none was abandoned. Thirty of 168,291 successful queries in the window took longer than 1 s (at most 8.2 s, against a 30 s deadline); 25 of them started in the two minutes around a Defender security-intelligence and platform update, and two within 7 s of the end of a Defender quick scan ten minutes later, both logged on the host. The `Winmgmt` host grew 0.182 MB/h over the window, with 30-minute medians moving between 13.1 and 16.9 MB without a trend after the first hour. The NetworkService host, now with seven providers, grew more than on the Windows 10-based host: by thirds 2.33, 0.34 and 0.54 MB/h (30-minute medians 15.0 to 18.8 MB), most of it in the first two hours. In the 72-hour run (interim, first 32 h) it continued to rise slowly, to 19.0 MB at 24 h and 19.8 MB at 28–32 h, while the `Winmgmt` host stayed between 12.5 and 16.1 MB without a trend. The client's private bytes grew 0.218 MB/h [0.153, 0.282] (0.37 B per array read): distinguishable from zero, but 4.6× below our pre-set bound and small in absolute terms (27.5 to 29.0 MB over the arm). The growth came in steps, not as a slope: in this run and in a replicate on the same host (0.230 MB/h [0.190, 0.271], before a change of virtual CPU model) the largest steps,

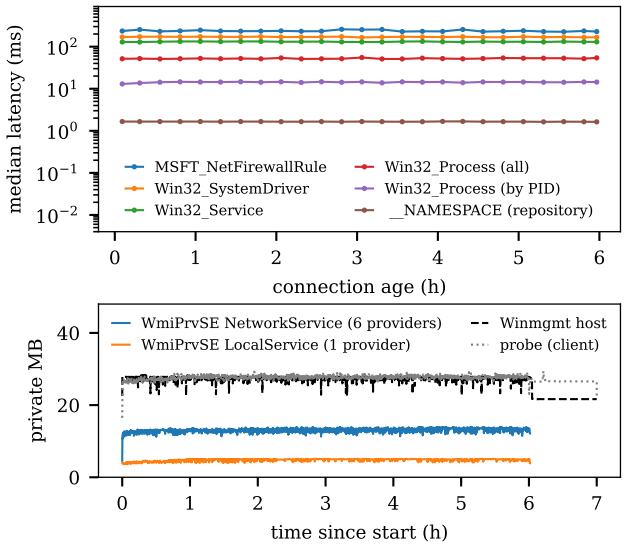


Figure 1: ST-1, Windows 10-based host. Top: median latency per 15-minute window against connection age for six query keys. Bottom: private bytes of the two provider hosts (labelled by account and number of providers; the providers are listed in Section 3), the Winmgmt host and the client; connections were released at 6 h. The Windows 11-based client is in Figure 2.

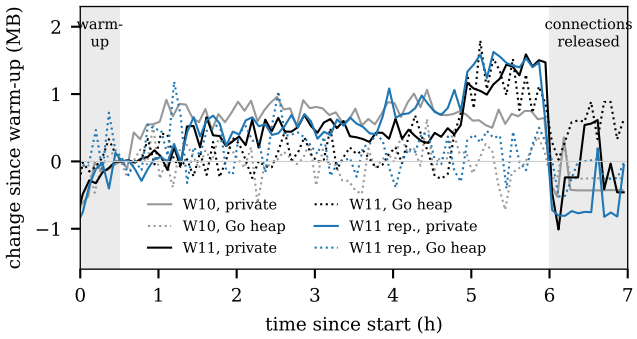


Figure 2: Client memory in ST-1 on both hosts: change in private bytes (solid) and Go heap (dotted) since the end of warm-up, 5-minute medians. "W11 rep." is a replicate run on the Windows 11-based host before its virtual CPU model was changed. Connections were released at 6 h.

0.4–0.6 MB each, fell at the same offsets from the start of the arm (1.37 h and 4.95 h), and private bytes were flat between them. The two runs share the probe build, rate and seeded schedule, so the steps follow the probe's own workload rather than host events. Go heap allocation rose with the steps, so part of the growth is in the Go runtime's heap; the ledger accounts for `VARIANT`s, so its clean result rules out a leaked `SAFEARRAY` or `BSTR`, not all native allocation, and we did not instrument the rest. All of it was reclaimed when the connections were released: at the end of the release test the client was 0.46 MB below its post-warm-up level (Figure 2). The 72-hour run on this host settles whether it plateaus: in its first 32 h the client rose from 27.3 to 28.6 MB by 10 h (about 1.0 MB of it Go heap) and was flat thereafter (0.013, 0.014 and 0.000 MB/h over 6–12, 12–24 and 24–32 h). Because closing a connection reclaims this memory, the result can be read as an argument for periodic reconnection. It is a weak one. Even if the growth were linear in queries and never levelled off, at the production rate it would amount to about 4.4 kB/h, or 3.2 MB per month of connection age, so reconnecting monthly would bound it there; our earlier client's 6-hour cycle reconnects about 120 times as often, and Section 7 shows that a timer recovers from the one fault that does break a connection only when it next fires (30 min in our runs), against seconds for reconnecting on the error.

Providers reload anyway. On both hosts, read from the `__CacheControl` and `__ProviderHostQuotaConfiguration` instances, `ClearAfter` is 30 s (15 s for event sinks) and the provider-host quotas are identical (512 MB per host, 1 GB for all hosts, 4,096 handles, 256 threads, 32 processes): neither is tuned per architecture. At the production rate ($k = 1$) most classes are queried less often than that, and on the Windows 10-based host the WMI-Activity log showed six providers reloading every 1–2 minutes *inside* one long-lived host PID (event 5857 within 10 minutes); the Defender provider's host itself was replaced, and idle hosts exited 60–65 s after release (60 s on the Windows 11-based host). A production client thus gets a free refresh of provider state every few minutes; only its connection and the WMI service age.

5 What degrades is the client

Unreleased arrays (our earlier client, pull request 115). Property values read through `Properties_` that were never passed to `VariantClear` leak their `SAFEARRAY`s. In long soak runs of the earlier client this was steady memory growth (+1.45 MB/h over 13.5 h on a Windows 10 VM, +2.29 MB/h over 20 h on a laptop) that scaled with the firewall collector's frequency (+8.8 ± 0.6 MB/h at 60 s, +0.44 ± 0.52 after the fix) and that *recycling did not cure* (2.5 MB/h with recycling working): the leak is in the client. Our positive control re-enables the bug in the probe. On the Windows 10-based host at $k = 10$ for 1 h (3,180 queries and 31,650 array reads in the 30-minute measurement window) the client's private bytes grew 8.97 MB/h [8.82, 9.10], i.e. 141.5 B per array read (95% CI [139.3, 143.6]), against 0.06 B for the correct client (Table 1); the probe's ledger flagged 780 `VARIANT` values acquired and never released in the window (about 5.7 kB per retained value, consistent with the firewall-rule `SAFEARRAY`s). On the Windows 11-based host we ran the control at two rates, $k = 5$ and $k = 10$: per array read the leak was 126.5 B [120.0, 132.9] and 115.8 B [109.1, 122.6], with overlapping CIs at the two rates and 11–18% below the Windows 10-based value, while the hourly growth nearly doubled (7.40 to 13.63 MB/h); the replicate battery on that host gave 139.4 and 136.2 B. The leak therefore tracks reads, not connection age, in our controlled runs as well as in the earlier client's frequency sweep, and recycling did not release it.

Cross-apartment calls (pull request 118). Our earlier client called the apartment-threaded scripting objects from multi-threaded-apartment goroutines, so COM marshalled every call into a single host STA (confirmed in debugger stacks). Moving each connection to its own STA cut the wall-clock time per collection cycle about 3× (process poll 1.31–1.40 s to 0.42–0.44 s; firewall rules 8.5 s to 2.8 s) and steady-state CPU on a busy 16-CPU laptop from 28.3% to 3.5–3.9% of a core (a 5–8× reduction), eliminating 1,490 abandoned calls in 69 h (≈21/h). On a quiet 2-vCPU VM the same bug cost 5.0% vs 0.9% and caused no abandoned calls: the symptom scales with host load, which makes it look like ageing on busy machines. This cost is specific to clients that drive the apartment-threaded scripting objects across apartments; native `IWbemServices` clients and clients on the newer Management Infrastructure (MI) API [17] do not marshal through a single STA and would not show it. Pull requests 115 and 118 are in esinta's private repository; their descriptions and measurements are archived with the artifact [1].**[the archived extract (docs/prs/) is pending the authors' review before it is published]**

6 Silent failures

A library that swallows errors. In `go-ole` v1.3.0 [10], `oleutil.ForEach` (`oleutil.go`, lines 118–126) enumerates an object's `_NewEnum` through `IEnumVARIANT` and loops while the number of items returned is positive:

```
118  for item, length, err := enum.Next(1); length > 0;
119      item, length, err = enum.Next(1) {
120    if err != nil { return err }
121    ...
122  }
123  return nil
```

When `Next` fails with zero items, the loop condition is false, the error is never inspected and the function returns `nil`. The same gap truncates a result silently when `Next` fails after some items have been returned: those items are delivered, the terminating error is dropped, and the caller sees a short but "successful" enumeration. On both hosts, `SELECT * FROM` a non-existent class fails only at `Next` with `WBEM_E_INVALID_CLASS` (0x80041010); `ForEach` reports success with 0 items, while our strict loop records the error. The truncating case occurred unprovoked: on the Windows 11-based host, while a Defender platform update replaced the Defender provider's host, six Defender queries in 10 s failed at `Next`, four with `WBEM_E_FAILED` (three of them after returning one row) and two with `WBEM_E_PROVIDER_LOAD_FAILURE`; a client using `ForEach` would have recorded three one-row and three empty successes.

Production queries that never worked. Three of the earlier client's 23 queries fail at `Next` on every call, with the same codes on both architectures (Table 2). On the Windows 10-based host, in the 7 h before we stopped the client there to begin this study, WMI-Activity recorded

Table 2: Silent failures. All return success to a lenient client. Codes are the strict loop's, at `Next`, on every call.

Query / class	Win 10-based	Win 11-based
`SELECT Name FROM __EventConsumer`	`0x80041017`	`0x80041017`
`SELECT InstanceID, Program, Package FROM MSFT_NetApplicationFilter`	`0x80041017`	`0x80041017`
`... FROM MSFT_ScheduledTaskInfo`	`0x80041010`	`0x80041010`
`Win32_SystemDriver` with any configuration property (or `*`)	333 of 334; `vwifibus` (binary absent) omitted	all (384, later 385; no driver lacks its binary)

84 `5858` errors for each, while the client's own result log recorded a 0-row, 0 ms success at each of their positions in every cycle (the log has no query text, so attribution is by position). From the client's code, the consequences are concrete: the firewall collector records no application path, non-standard event consumers are missing from the persistence inventory, and scheduled tasks lack last-run and next-run times. Each was a plausible-looking query. `Name` is not a property of the abstract `__EventConsumer` base class (which has `CreatorSID`, `MachineName` and `MaximumQueueSize`); it appears only on the five derived consumer classes [21]. `MSFT_NetApplicationFilter` exposes `AppPath` and `Package` but no `Program` (class definition read on-host with `Get-CimClass`); "Program" is the name of the `Set-NetFirewallApplicationFilter` cmdlet parameter [22], the likely origin of the mistake. `MSFT_ScheduledTaskInfo` is not a WMI class; the per-task run times are exposed through `MSFT_TaskDynamicInfo` (`LastRunTime`, `NextRunTime`, `LastTaskResult`, `NumberOfMissedRuns`; class definition read on-host with `Get-CimClass`) [23].

A provider that drops rows. On the Windows 10-based host `Win32_SystemDriver` returned 333 drivers whenever the query selected any configuration property (`DisplayName`, `PathName`, `StartMode`, `StartName`, `ServiceType`, `Description`, or `*`), and 334 for `SELECT Name` or `SELECT Name, State`. The missing driver, `vwifibus`, is registered with the service control manager but its image path points to a file that does not exist. No error is returned at any layer; only the comparison with the service control manager reveals it. A provider might reasonably omit a driver whose image cannot be read; what makes this a silent failure is that the row count depends on the projection—present for `SELECT Name`, absent once any configuration property is requested—with no error or other signal at any layer. On the Windows 11-based host the same driver is registered with its binary present in the driver store, and `Win32_SystemDriver` returned every registered driver (384, later 385) for every selection. The omission therefore depends on host state, a registered driver whose binary is missing, which is common after

Table 3: E-fault, 90 min per arm. Errors caused by the host kill and by the Winmgmt restart (the 60 min after it holds 60,000 and 30,600 queries at the two hosts' rates); a call in flight when a fault began is counted with that fault (Windows 11-based host only: one call at each kill, two or three at each restart); recovery is the time until every connection that failed has answered again.

		Windows 10-based, $k = 100$			Windows 11-based, $k = 50$		
Arm	Strategy	kill errors	restart errors	recovery	kill errors	restart errors	recovery
E0	none	0	59,995	none in 60 min	1	30,602	none in 60 min
E1	timed (1 h)	0	30,002	1,804.5 s	1	15,312	1,811.5 s
E2	adaptive	0	8	6.0 s	1	10	10.0 s
E3	Earlier-client pool	0	59,995	none in 60 min	1	30,603	none in 60 min

driver removal, rather than on the Windows architecture. Because it is deterministic, we treat it as a per-key truth baseline and count S2 only above it.

Races are not failures. In 19,716 calibration queries the only other flags were three scheduled-task counts one short as the count changed and one process that exited during its query: timing, not WMI, separable only by bracketing truth before and after each query. One case is not explained by timing: in the Windows 11-based ST-1, three consecutive scheduled-task results in 12 s each lacked the same task, although the truth snapshots before and after each query contained it (S2, 261 of 262 rows); later results were complete. We count these three as silent failures (Table 1) and have no explanation for them.

7 Fault recovery and refresh

If connections do not age, refresh is only worth its cost as recovery from faults. On each host we ran four 90-minute arms back to back ($k = k_H$, fresh connections each), one per strategy: *none* (E0); *timed*, redial every connection after 1 h (E1); *adaptive*, retire a connection when a failed call's effective error code is connection-level and redial at its next query (E2); and a replica of our earlier client's *pool*, which recycles at 6 h, 50,000 queries or 5 min idle, or when the outer `HRESULT` maps to access denied, transport failure, "not available" or COM not initialised (E3; rules copied from the earlier client's source). At 15 min we killed the `WmiPrvSE` host serving `Win32_Process` and its co-hosted providers; at 30 min we restarted `Winmgmt` and its running dependent services through the service control manager (1.9 s in total on the Windows 10-based host, which had two running dependents; 0.2 s on the Windows 11-based host, which had none). On the Windows 11-based host a replicate battery, run before its virtual CPU model was changed, gave the same outcomes in every arm (recovery 1,811.5 s timed, 11.5 s adaptive, none for the other two). The three deterministically failing keys are excluded.

A provider-host crash cost at most the call in flight. On the Windows 10-based host each of the four kills landed between queries and no query failed: the connection is to `Winmgmt`, which had a replacement host with the same providers running 31 ms later (E0), and the 99th-percentile latency in the following 30 s was unchanged (225 vs. 231 ms before). On the Windows 11-based host, where queries take longer, each kill landed while three or four calls served by the killed host were in flight, at the same point of the schedule in all four arms. A firewall-address query that had started 21–32 ms earlier and was still waiting for its first object failed 12–42 ms after the kill, at `IEnumVARIANT::Next`, with `RPC_S_CALL_FAILED` (0x800706BE); this agrees with Microsoft's guidance that when WMI stops a host for a quota violation the query it is serving fails [15]. The calls that were already returning rows completed with full results matching ground truth: a driver query started about 245 ms earlier (all 385 drivers), a scheduled-task query, and in three arms the process list. The connections were unaffected: under no refresh, the timer and the pool the next query on each succeeded and no other query failed before the service restart 15 min later, and the 99th-percentile latency in the 30 s after the kill was within 2% of the 5 minutes before (762 vs. 747 ms, E0). The adaptive client treats `RPC_S_CALL_FAILED` as connection-level, so it retired the healthy connection and redialled in 15 ms. The failure surfaces at `Next`, the call whose result the `ForEach` helper of Section 6 discards, so a client using it would have recorded an empty result as a success. These are four kills at one offset into a call, not a map of every phase of a call, and all were external kills; we did not provoke a quota kill by lowering `MemoryPerHost`.

A service restart is permanent. After `Winmgmt` restarted, every call on every existing connection failed with `RPC_S_SERVER_UNAVAILABLE` (0x800706BA) and none recovered on its own (Windows 10-based E0: 59,995 of 60,000; the 5 successes were calls issued in the 175 ms before the service stopped; Windows 11-based E0: 30,602, including two calls in flight when the restart began, which failed with `RPC_S_CALL_FAILED` or `RPC_S_UNKNOWN_IF` (0x800706B5) after returning 21 and 158 rows; a third call in flight completed; in the replicate battery, calls in flight also failed with `RPC_E_DISCONNECTED` (0x80010108)). The timed strategy was down until its timer fired, 30 minutes later; an outage of up to one refresh interval is inherent to it. This is not peculiar to our client: Microsoft documents that its Configuration Manager client (`CcmExec`) is left non-functional after the WMI service is restarted, because WMI cancels outstanding requests and does not re-establish them, and recovers only

when a periodic health check restarts the client [14]. The adaptive client lost exactly one query per connection: each of its 9 connections on the Windows 10-based host failed once, was retired, redialled in 1–6 ms, and its next scheduled query succeeded; Table 3 counts 8 because one of the nine failures fell on an excluded deterministic key. On the Windows 11-based host all 10 connections failed once (three on calls in flight when the restart began) and redialled in 1–55 ms. The recovery time, 6.0 and 10.0 s, is set by the least-busy namespaces: on the Windows 11-based host the Device Guard connection, queried every 6 s at this rate, failed on its first query after the restart, 4.0 s in, and its next query succeeded at 10.0 s; at $k = 1$ the same pattern would take up to two production polling intervals. Our earlier client's pool behaved exactly like no refresh on both hosts (59,995 and 30,603 errors, no recycle): `RPC_S_SERVER_UNAVAILABLE` is not an outer code its health check treats as unhealthy, and at arm end its connections were 1.5 h of 6 h old with at most 21,000 (10,500 on the Windows 11-based host) of 50,000 queries. By its rules the first recycle would come from the query budget at about 214 min at $k = 100$, from the 6 h age limit at $k = 50$ and at the production rate; we did not run that long. The refresh mechanism did not fire on the one fault that needed it, on either host.

8 Discussion

Guidance. (1) Make the client correct first: one STA per connection, all COM calls on it, every `VARIANT` cleared, and a test that runs a few hundred thousand queries and checks for growth; timed recycling hides neither of the bugs above. (2) Never let an enumeration end without inspecting the last `HRESULT`; audit libraries for this pattern. (3) Validate each query once at start-up and verify results against an independent source where one exists; "0 rows" is a claim, not a fact. (4) Reconnect on connection-level errors by inspecting the effective error code, not only the outer `HRESULT`; test the health check by restarting `Winmgmt`. In our fault injection the service restart produced `RPC_S_SERVER_UNAVAILABLE` (observed); a robust health check should additionally treat `WBEM_E_TRANSPORT_FAILURE` and `WBEM_E_NOT_AVAILABLE` as connection-level (documented [18], not observed here), and `RPC_E_DISCONNECTED` and `RPC_S_UNKNOWN_IF` (each observed on calls in flight when the service stopped). `RPC_S_CALL_FAILED` from a crashed provider host left the connection usable; retiring on it costs one redial. A timer adds churn and leaves an outage of up to one interval. (5) Do not attribute `WmiPrvSE` growth to your client without per-host attribution and a foreign-client meter.

Threats to validity. Two hosts, one per architecture, and one workload; each fault was injected once per strategy and host, so the fault results are existence proofs of a strategy's behaviour, not rates. The two hosts also differ in acceleration (the Windows 10-based ran at $k = 100$, the Windows 11-based at $k = 50$, forced by the 40% CPU budget) and in RAM, so architecture and rate move together; the between-host difference in client and provider-host memory growth cannot be attributed to the operating system alone. The CPU model does not explain the lower rate: the probe's induced CPU per query on the Windows 11-based host was about twice that on the Windows 10-based host both before and after we gave it the same virtual CPU model (the Windows 11-based results reported here are from after the change; a replicate from before it agrees). Accelerating the query rate tests count, not age, and not the provider unload/reload cycle, which at k_H does not occur: our longest correct-client connections are 6 h old; the 7-day run is in progress on the Windows 10-based host and 72 h old on the Windows 11-based host. The ETW meter ran one minute in fifteen in every arm, so foreign activity is sampled; we count a window only if the meter recorded the probe's own operations in it and lost no events (a stale session left by an earlier, killed probe process once made later sessions drop every event under load; those runs are not reported). The pre-registered rule (an arm is invalid when more than 10% of its windows exceed 60 foreign operations per hour) was met by every self-test and client-bug arm and by the four Windows 11-based fault arms, which recorded no foreign operations, but not by the four Windows 10-based fault arms. In the self-test and client-bug arms one window was flagged, in the Windows 10-based ST-1 (1 of 24: 6 operations from a system service, most likely the Windows Update service by timing). Each Windows 10-based fault arm had one flagged window of six, in all four the window containing the service restart: 38 operations in the 10 s after it, from two system service hosts querying `HNet_ConnectionProperties` and `Win32_ComputerSystemProduct`, consistent with the two dependent services the restart also restarts (not confirmed by PID). Excluding that window, which our own fault induced, all four would meet the rule; since the exclusion was not pre-registered, we report the four arms as a declared deviation. Their outcomes are the error codes and recovery times of our own connections, and they agree with the Windows 11-based arms. The earlier client's degradation numbers come from before/after comparisons of single changes across our soak-test hosts (a VM and a laptop). Our no-degradation result is for a client whose COM usage is correct on this provider set; providers themselves can leak, as in a documented `WmiPrvSE` user-profile case [16], which per-host attribution (guidance 5) is designed to separate from client growth. Older and server-only architectures, and further client builds, remain future work.

Related work. Microsoft documents the mechanisms we rely on: provider hosting and unloading [2, 3], quotas [5], enumeration semantics [6] and COM apartment rules [7]. The newer Management Infrastructure (MI) API [17] is the client surface the CIM cmdlets use and a natural target for replication; our findings here are for a COM/scripting client, and whether they transfer to MI and to native `IWbemServices` clients is open. We are not aware of published measurements of WMI connection ageing under a verified-correct client, nor of a catalogue of WMI silent failures; we welcome point-

ers.

9 Conclusion

Long-lived WMI connections, used correctly, did not degrade in latency, median or tail, on either architecture; the client's memory grew by at most 1.5 MB over 6 h, all of it returned when the connections closed, and the provider hosts by up to 4 MB on the Windows 11-based host and less than 1 MB on the Windows 10-based host, with the 72-hour and 7-day runs deciding whether either levels off; the symptoms we could reproduce were produced by the client, and the costliest failures were silent. Refresh is a recovery mechanism, best driven by errors. The probe, catalogue, analysis code and run logs are in the public repository [1], with the raw run data as a release.

References

[1] esinta. *wmi-degradation-research*: probe, analysis, run logs and paper source. https://github.com/esinta/wmi-degradation-research

[2] Microsoft. *Provider Hosting and Security*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/provider-hosting-and-security

[3] Microsoft. *Unloading a Provider*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/unloading-a-provider

[4] Microsoft. *__CacheControl class*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/--cachecontrol

[5] Microsoft. *__ProviderHostQuotaConfiguration class*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/--providerhostquotaconfiguration

[6] Microsoft. *IEnumWbemClassObject::Next*. https://learn.microsoft.com/en-us/windows/win32/api/wbemcli/nf-wbemcli-ienumwbemclassobject-next

[7] Microsoft. *Single-Threaded Apartments*. https://learn.microsoft.com/en-us/windows/win32/com/single-threaded-apartments

[8] Microsoft. *VariantClear function*. https://learn.microsoft.com/en-us/windows/win32/api/oleauto/nf-oleauto-variantclear

[9] Microsoft. *Tracing WMI Activity*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/tracing-wmi-activity

[10] go-ole contributors. *go-ole v1.3.0, oleutil/oleutil.go*. https://github.com/go-ole/go-ole/blob/v1.3.0/oleutil/oleutil.go

[11] W. K. Newey and K. D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708, 1987.

[12] StackExchange. *wmi: Go WMI package*, README warning on the memory bug. https://github.com/StackExchange/wmi

[13] Zabbix. *ZBX-8506: wmi.get on Windows agent causes memory leak (unreleased pclsObj)*. https://support.zabbix.com/browse/ZBX-8506

[14] Microsoft. *SMS Agent Host service isn't automatically restarted after the WMI service restarts*. https://learn.microsoft.com/en-us/troubleshoot/mem/configmgr/client-management/sms-agent-host-not-automatically-restarted

[15] Microsoft. *Scenario guide: troubleshoot WmiPrvSE quota-exceeded issues*. https://learn.microsoft.com/en-us/troubleshoot/windows-server/system-management-components/scenario-guide-troubleshoot-wmiprvse-quota-exceeded-issues

[16] Microsoft. *Memory leak occurs in the Wmiprvse.exe process in Windows 7 or Windows Server 2008 R2* (KB2832248). https://support.microsoft.com/en-us/topic/067c6067-0150-ce7e-3c8a-ba762a0b1168

[17] Microsoft. *Management Infrastructure (MI) reference*. https://learn.microsoft.com/en-us/windows/win32/api/mi/

[18] Microsoft. *WMI Error Constants*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/wmi-error-constants

[19] Microsoft. *IEnumVARIANT::Next method*. https://learn.microsoft.com/en-us/windows/win32/api/oaidl/nf-oaidl-ienumvariant-next

[20] Microsoft. *Service Host grouping in Windows 10*. https://learn.microsoft.com/en-us/windows/application-management/svchost-service-refactoring

[21] Microsoft. *__EventConsumer class*. https://learn.microsoft.com/en-us/windows/win32/wmisdk/--eventconsumer

[22] Microsoft. *Set-NetFirewallApplicationFilter*. https://learn.microsoft.com/en-us/powershell/module/netsecurity/set-netfirewallapplicationfilter

[23] Microsoft. *Get-ScheduledTaskInfo* (outputs `MSFT_TaskDynamicInfo`). https://learn.microsoft.com/en-us/powershell/module/scheduledtasks/get-scheduledtaskinfo